# Best Model Explained

CS155 Miniproject 1 - predicting Spotify song popularity (High / Low) from audio features

| | |
|---|---|
| Best submission | **prediction_6.csv** - public leaderboard AUC **0.861** |
| Cross-validated AUC | 0.863 (3 x 5-fold grouped CV, blend weights scored with a nested split) |
| Model | Weighted rank-average of 8 models: XGBoost, LightGBM, CatBoost, ExtraTrees, RandomForest, SVM, MLP, k-NN |
| Features | 67 = 13 audio features + time-series features from the release date + "album look-alike" features + out-of-fold date target encodings |
| Code | time_features.py (features), cs155-miniproject1.py (tuning), ensemble_v2.py (final model) |
| Reproduce | python ensemble_v2.py --start 6 (uses the tuned params in best_params_v3.json) |

## 1. The big ideas

Five ideas account for nearly all of the improvement from the first tuned model (CV 0.832) to the final one (CV 0.863):

- **The release date is the strongest signal, not the audio.** Popularity depends heavily on *when* and *how* a song was released. The High rate falls from about 50% for pre-2010 songs to 25% for 2020s songs, and it changes a lot with the weekday (Tuesday 46% vs Wednesday 22%).
- **Release density acts as a stand-in for albums.** Counting how many songs (train + test) share the exact release date, month, and year captures album size and big release events. This was the single largest gain (+0.017 AUC).
- **Same-day look-alikes find the same artist or album.** Songs released on the same day that sound alike are probably from the same album. Counting them and measuring the distance to the nearest one added about +0.008.
- **Validation must respect duplicates.** About 220 songs appear more than once in train (same `uri`) and never in test. Cross-validation is grouped by song, so copies never land on both sides of a split and inflate the score.
- **Blend diverse models by rank.** AUC only depends on ordering, so each model's predictions are converted to ranks and combined with non-negative weights that maximise out-of-fold AUC.

## 2. Data and validation

- **train.csv**: 3,865 rows (1 has no label and is dropped -> 3,864), 35% High. **test.csv**: 863 rows.
- Columns: 13 audio features (danceability, energy, loudness, tempo, duration_ms, ...), the album release date, and three URL columns plus `type`.
- **Dropped columns:** `track_href`, `uri` and `analysis_url` all contain the same random 22-character Spotify ID (AUC 0.49 as a number, no test ID appears in train); `type` is constant. The ID is only used to define CV groups. Fetching the URLs would count as an additional data source, which the rules forbid.
- **Validation:** StratifiedGroupKFold with 5 folds, grouped by song ID, repeated 3 times with different seeds. Every training row gets an out-of-fold (OOF) prediction, and AUC is computed on those. Hyperparameters were tuned on a *different* fold seed than the one used for the final score, so the score is not inflated by tuning.
- **Test predictions** average the 15 fold models; the predictions never use test labels.

# 3. Features (time_features.py)

All features are built in `build_features(train, test, groups)`. They use no labels, and train and test are processed together, so counts see the whole release calendar. The gains below come from `feature_ablation.py`: tuned LightGBM, grouped 2 x 5-fold CV, adding one group at a time.

| Group | Features | Gain (LGBM AUC) | Used? |
|---|---|---|---|
| baseline | 13 audio + release_year, release_month, days_since_release, is_jan1 | 0.829 | yes |
| trend | continuous time (fractional year), log age, decade | +0.001 | yes |
| seasonality | weekday, day-of-year, week, quarter, day; sin/cos of month, weekday, day-of-year; Friday / Tuesday release-day flags, holiday (Nov-Dec), summer | +0.011 | yes |
| density | songs sharing the exact date / month / year; same date and same tempo | +0.017 | yes |
| albumnn | same-date songs within audio distance 1.5; distance to nearest same-date song | +0.006 | yes |
| albumnn2 | counts at radius 1, 2, 3; 2nd-nearest same-date distance; nearest look-alikes across all dates | ~0 alone, helps CatBoost | yes |
| groupagg | mean of each audio feature over the release-date group, and the song's deviation | ~0, +0.003 CatBoost with the album features | yes |
| precision | date given as full date / year-month / year only | +0.000 | no |
| detrended | audio features minus the median of nearby release years | +0.000 | no |
| density2 | release counts in 7 / 30 / 365-day windows, same week, rank within year | -0.000 | no |

The chosen groups are listed once in `FEATURE_GROUPS` at the top of time_features.py, and every script imports them from there.

### Out-of-fold target encodings (in ensemble_v2.py)

On top of those 67 label-free features, the final model adds five features that *do* use labels, computed so a row never sees its own label:

- `te_date`, `te_ym`: smoothed High-rate of training songs released on the same date / month (+ their counts).
- `knn3`: mean label of the 3 audio-nearest training songs released on the same date.
- Inside each CV fold, training rows get these from an inner 5-fold split, and validation / test rows get them from all of the fold's training rows - the same situation as train -> test.
- Effect: about +0.003 for the boosters on the earlier feature set, but roughly zero once the album features exist (they carry the same information without labels).

# 4. Models and the blend

Tree hyperparameters were tuned with Optuna (30 trials per model, TPE sampler, maximising grouped 5-fold CV AUC) in cs155-miniproject1.py and saved to best_params_v3.json. SVM, MLP and k-NN use fixed settings on quantile-normalised inputs.

| Model | OOF AUC alone | Weight in prediction_6 | Notes |
|---|---|---|---|
| XGBoost | 0.860 | 0.241 | 1,089 trees, depth 6, lr 0.033, column subsample 0.30 |
| LightGBM | 0.860 | 0.176 | 662 trees, 30 leaves, lr 0.017 |
| CatBoost | 0.860 | 0.192 | 1,110 iterations, depth 7, lr 0.020 |
| ExtraTrees | 0.856 | 0.279 | 400 trees, depth 19, min leaf 3; most different from boosters |
| RandomForest | 0.857 | 0.057 | 600 trees, depth 16, max_features 0.21 |
| MLP | 0.834 | 0.054 | 128-64 hidden units, early stopping |
| SVM (RBF) | 0.821 | 0.001 | C = 1 |
| k-NN | 0.792 | 0.000 | k = 75, distance-weighted |
| **Blend** | **0.863** | | weights learned on OOF ranks; nested CV gives the honest score |

**Why a rank blend?** AUC only cares about ordering, and the models output probabilities on different scales. Converting each model's predictions to ranks (0-1) before averaging makes them comparable. Weights are non-negative, sum to 1, and are found with Nelder-Mead from several starting points. To avoid fooling ourselves, the blend's score is computed with a **nested** split: weights are learned on 4/5 of the OOF predictions and scored on the other 1/5.

**Why neural nets are minor:** with ~3.9k rows and tabular features, gradient-boosted trees handle thresholds and mixed scales naturally; the MLP scores ~0.83 and gets ~5% weight - useful only for a little diversity.

## 5. How the score improved

| Step | CV AUC | Public LB |
|---|---|---|
| Tuned 6-model ensemble on audio + basic date features | 0.832 | - |
| + trend, seasonality, release density; retuned | 0.855 | 0.857 |
| + date target encoding, stacking, SVM / MLP / k-NN (prediction_1-5) | 0.855-0.856 | - |
| + album look-alike + group features; retuned (prediction_6-10) | **0.863** | **0.861** (prediction_6) |

The leaderboard has tracked CV within about 0.002, so CV is a reliable guide. The public leaderboard uses only part of the 863 test songs, so a single public score has roughly +/-0.02 of noise; choose final submissions by CV, not by small public differences.

### What did not help

- Scaling or log-transforming duration: tree models are invariant to monotone transforms; the other models already use a quantile transform.
- Era-detrended audio, date precision, windowed release counts (see the table in section 3).
- URL / ID columns: random IDs, no signal. Train row order and test ID order: no leakage (AUC 0.51).
- Logistic-regression stacking instead of the weighted blend: same score within 0.0003.

# 6. Code map

| File | What it does |
|---|---|
| time_features.py | Parses dates and builds every feature group; `FEATURE_GROUPS` picks the ones in use. Imported by all other scripts. |
| cs155-miniproject1.py | Tunes 6 models with Optuna (`--tune --trials 30`), evaluates them on fresh 3 x 5-fold CV, fits blend weights, writes predictions.csv, best_params_v3.json and blend_weights.json. |
| ensemble_v2.py | **The final model.** Adds OOF target encodings and SVM / MLP / k-NN, runs 3 x 5-fold CV with and without target encoding, and writes five submissions (`--start 6` -> prediction_6..10.csv). Saves all OOF / test predictions to oof_v2_from6.npz. |
| feature_ablation.py | Experiment: CV AUC of LightGBM and CatBoost for any combination of feature groups, e.g. `python feature_ablation.py trend,seasonality,density`. |
| te_experiment.py | Experiment: which target encodings help. |
| refit_full.py | Refits the main-script ensemble on 100% of train (predictions_full.csv). |
| best_params_v3.json | Tuned hyperparameters for the current feature set (used by ensemble_v2.py). |
| *_log.txt | Output of each run: tune_log_v3.txt, ensemble_v3_log.txt, ... |

# 7. Code walkthrough

## 7.1 time_features.py - building features

Train and test are stacked so every count sees the full calendar, then split back apart at the end:

```
def build_features(tr, te, groups=...):
    al = pd.concat([tr, te], ignore_index=True)          # train rows first, then test
    raw = al['track_album_release_date'].astype(str)
    d = parse_date(raw)                                  # handles YYYY-MM-DD, YYYY-MM, YYYY
    F = al[AUDIO].copy()
    ...
    if 'density' in groups:
        F['n_same_date'] = raw.map(raw.value_counts())   # ~ album size / release event
        ...
    n = len(tr)
    return F.iloc[:n].reset_index(drop=True), F.iloc[n:].reset_index(drop=True)
```

The album look-alike features compare songs only within the same release date. Audio is standardised; for each date group, a pairwise distance matrix gives the count of close songs and the nearest distance:

```
for _, idx in al.groupby(raw).indices.items():          # rows sharing one release date
    D = np.linalg.norm(Zs[idx, None, :] - Zs[None, idx, :], axis=2)
    np.fill_diagonal(D, np.inf)                          # ignore the song itself
    n_close[idx] = (D < 1.5).sum(axis=1)                 # same-day songs that sound alike
    nn_dist[idx] = D.min(axis=1)                         # how close the nearest one is
```

## 7.2 cs155-miniproject1.py - tuning

- **Section 1**: drop the unlabeled row; `groups = factorize(uri)` for grouped CV.
- **Sections 6-7**: `make_model(name, params)` builds each model; `suggest(name, trial)` is the Optuna search space. Each trial scores grouped 5-fold CV AUC (seed 42). Best params are saved to best_params_v3.json; delete a model's entry or pass `--tune` to retune.
- **Section 8**: re-evaluates every tuned model on fresh folds (seeds 2026-2028), producing OOF and fold-averaged test predictions.
- **Section 9**: `fit_weights` finds blend weights; the nested estimate decides which blend is used.

### 7.3 ensemble_v2.py - the final model

All rows live in one feature table XA: indices 0..n-1 are train, n..n+m-1 are test, so encodings can be applied to any set of rows.

```
def matrices(fit_idx, apply_sets, use_te):
    Xf = XA.iloc[fit_idx]                              # rows the model is trained on
    Xs = [XA.iloc[a] for a in apply_sets]              # rows to predict (validation, test)
    if use_te:
        Xf += encode_oof(fit_idx)                      # inner 5-fold: no row sees its own label
        Xs = [x + encode(fit_idx, a) for x, a in ...]  # fit on all fit_idx labels
    return Xf, Xs
```

The main loop: for each of the 15 grouped folds, build the matrices, fit all 8 models, store validation predictions (OOF) and add test predictions to a running average:

```
for k, (a, b) in enumerate(splits):                    # 3 repeats x 5 grouped folds
    Xf, (Xb, Xt) = matrices(a, [b, test_idx], use_te)
    for mdl in TREES + ALT:
        pb, pt = fit_predict(mdl, Xf, y[a], [Xb, Xt], seed=k)
        oof[(mdl, use_te)][b] += pb / N_REP             # each row is validated 3 times
        tst[(mdl, use_te)]    += pt / len(splits)       # test = mean of 15 fold models
```

prediction_6 is the first submission written:

```
O_te = column_stack([rank01(oof[r]) for r in te_runs])    # OOF ranks, 8 models
T_te = column_stack([rank01(tst[r]) for r in te_runs])    # test ranks
w1 = fit_weights(O_te, y)                                 # maximise OOF AUC
write('prediction_6.csv', T_te @ w1, nested(te_runs, 'weights'))
```

| Output | Method | CV AUC |
|---|---|---|
| prediction_6 | weighted rank blend, 8 models, with target encoding **(best on leaderboard)** | 0.8629 |
| prediction_7 | logistic-regression stacker on the same OOF predictions | 0.8627 |
| prediction_8 | plain rank average of the 5 tree models | 0.8629 |
| prediction_9 | weighted blend without target encoding | 0.8630 |
| prediction_10 | prediction_6's blend refit on 100% of train (3 seeds per model) | ~0.8629 |

## 8. How to reproduce

```
cd project
python cs155-miniproject1.py               # loads best_params_v3.json (add --tune --trials 30 to retune, ~1 h)
python ensemble_v2.py --start 6            # ~25 min -> prediction_6.csv ... prediction_10.csv
python feature_ablation.py trend,seasonality,density,albumnn   # optional: test a feature set
```

Requirements: pandas, numpy, scikit-learn, xgboost, lightgbm, catboost, optuna, scipy.

## 9. Notes for the report

- **Rules:** only train.csv / test.csv are used. The density and look-alike features read the test set's *features* (release dates and audio), never its labels - a standard transductive technique. If the TAs consider this out of bounds, compute them from train.csv only (a small loss of signal).
- **Top-10 features question:** run SHAP or permutation importance on the boosters; expect release year / age, same-date counts, weekday, duration and the look-alike counts near the top.
- **Extra-credit idea:** Spotify's audio-analysis endpoint (the dropped analysis_url) contains segment-level timbre and pitch timelines - a natural "additional data" answer.

- **LLM usage:** this pipeline was developed with Claude (Claude Code); report it in the LLM-usage section as the assignment requires.